

Calcolatori Elettronici

15 giugno 2026

Teoria

Esercizio 1 (7 punti): Sia $9X.Y$ un numero decimale, in cui X e Y sono, rispettivamente, il penultimo ed ultimo numero della propria matricola—occorre sostituire le cifre della matricola a X e Y (7 e 8), non eseguire una moltiplicazione. Calcolare la rappresentazione in virgola mobile, utilizzando 1 bit di segno, 3 bit di esponente, 5 bit di mantissa, dettagliando tutti i passi del procedimento.

Esercizio 2 (8 punti): Un distributore automatico accetta tre tipi di monete del valore di 5, 10 e 20 centesimi. La merce contenuta ha un costo pari a 15 centesimi e viene erogata non appena si raggiunge un importo uguale o maggiore. Il distributore prevede la possibilità di erogare un resto massimo di una moneta da 5 centesimi ed una moneta da 10 centesimi. Realizzare la rete sequenziale che governa il funzionamento di tale distributore. Determinare l'equazione di eccitazione minimizzata di un flip flop di stato a scelta e disegnarne il relativo circuito equivalente.

Esercizio 3 (8 punti): Si consideri una cache ad accesso diretto con una capacità totale di 64 KB e linee da 64 Byte ciascuno. L'architettura adotta un indirizzamento della memoria a 32 bit orientato al byte. Determinare la suddivisione dei 32 bit dell'indirizzo logico nei tre campi: Offset, Indice e Tag. Successivamente, calcolare a quale linea di cache viene associato l'indirizzo esadecimale `0x001A4F3C`.

Esercizio 4 (7 punti): Discutere il funzionamento del *carry lookahead adder*, illustrando per quale motivo tale implementazione di sommatore è preferibile rispetto ad un *full adder*, quando le parole binarie da sommare sono composte da un numero elevato di bit.

Soluzioni

Esercizio 1

Assumendo $X = 7$ e $Y = 8$, il valore numerico da convertire risulta essere 97.8. Il procedimento segue la struttura dello standard IEEE 754 applicato a un formato di 9 bit.

Poiché il numero è positivo, il bit di segno è impostato a 0.

La conversione della parte intera $(97)_{10}$ avviene tramite divisioni successive per 2. Nello specifico:

- $97/2 = 48$ con resto 1
- $48/2 = 24$ con resto 0
- $24/2 = 12$ con resto 0
- $12/2 = 6$ con resto 0
- $6/2 = 3$ con resto 0
- $3/2 = 1$ con resto 1
- $1/2 = 0$ con resto 1

Leggendo i resti al contrario, si ottiene $(97)_{10} = (1100001)_2$.

La conversione della parte frazionaria $(0,8)_{10}$ avviene tramite moltiplicazioni successive per 2, estraendo la parte intera ad ogni passaggio. Si procede calcolando:

- $0,8 \times 2 = 1,6$ (parte intera 1)
- $0,6 \times 2 = 1,2$ (parte intera 1)
- $0,2 \times 2 = 0,4$ (parte intera 0)
- $0,4 \times 2 = 0,8$ (parte intera 0)

Il numero presenta una periodicità nella sua rappresentazione binaria. Unendo le parti intera e frazionaria, il numero in base 2 è $(1100001.1100)_2$.

Procediamo con la normalizzazione spostando la virgola affinché rimanga una sola cifra significativa (pari a 1) alla sua sinistra. Si ottiene $1,1000011100 \times 2^6$.

Il valore dell'esponente reale è $E = 6$. Per calcolare l'esponente polarizzato, utilizziamo la formula $E_{biased} = E + bias$. Con 3 bit per l'esponente, il bias è calcolato come $2^{3-1} - 1 = 3$. Di conseguenza, $E_{biased} = 6 + 3 = 9$.

Tuttavia, il valore dell'esponente polarizzato calcolato (9) supera il valore massimo consentito per 3 bit (7). Di conseguenza, il numero risulta troppo grande per essere codificato nel formato richiesto, rendendo impossibile completare la rappresentazione in virgola mobile secondo il formato specificato.

Esercizio 2

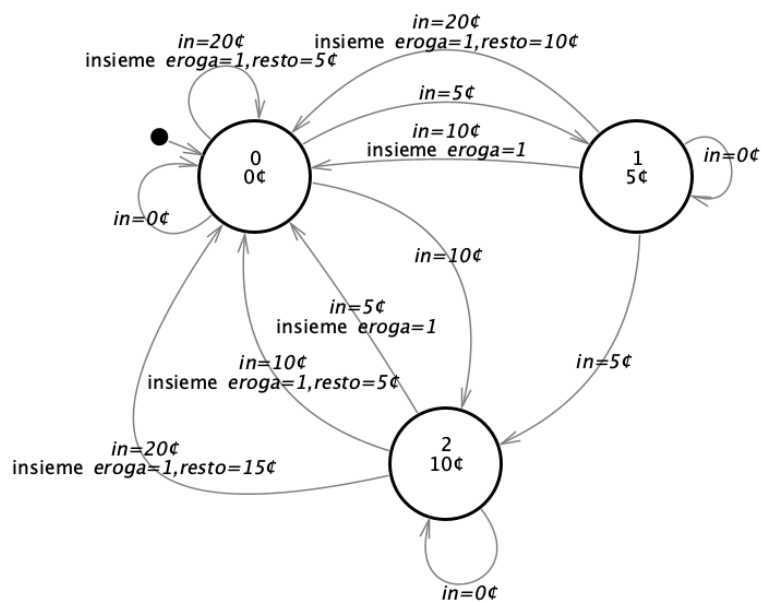
Si può costruire un automa in cui gli stati rappresentano il credito che ha accumulato l'utilizzatore del distributore automatico. Si può utilizzare quindi la seguente codifica degli stati:

credito	y_1	y_0
0€	0	0
5€	0	1
10€	1	0

Analogamente, in ogni istante di campionamento dell'input, l'utente sta inserendo nel distributore una delle monete consentite o nessuna moneta. Gli input possono quindi essere codificati come:

moneta	x_1	x_0
0€	0	0
5€	0	1
10€	1	0
20€	1	1

Il sistema ha in uscita 3 possibili segnali di controllo: *eroga*, per indicare che è necessario erogare il prodotto; *resto₅* per indicare che è necessario erogare una moneta da 5€ come resto; *resto₁₀* per indicare che è necessario erogare una moneta da 10€ come resto. L'automa può quindi essere rappresentato come segue:



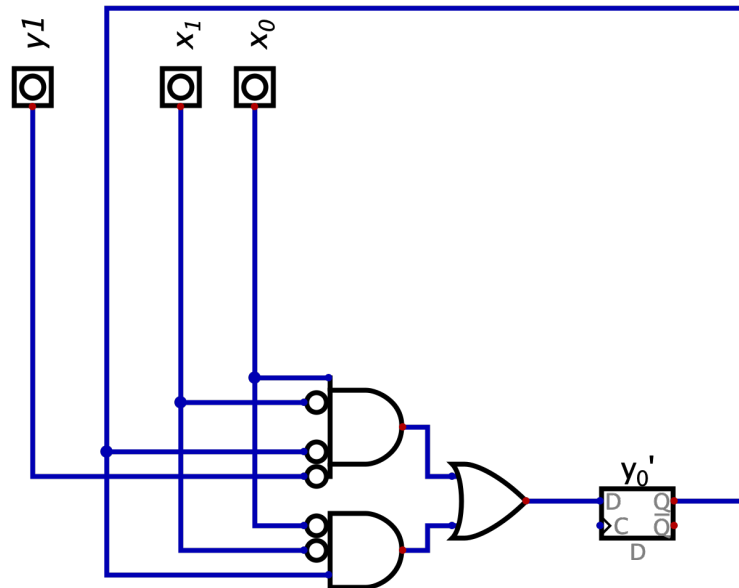
La tabella degli stati e transizioni che si ottiene è la seguente:

y_1	y_0	x_1	x_0	y'_1	y'_0	$eroga$	$resto_5$	$resto_{10}$
0	0	0	0	0	0	0	0	0
0	0	0	1	0	1	0	0	0
0	0	1	0	1	0	0	0	0
0	0	1	1	0	0	1	1	0
0	1	0	0	0	1	0	0	0
0	1	0	1	1	0	0	0	0
0	1	1	0	0	0	1	0	0
0	1	1	1	0	0	1	0	1
1	0	0	0	1	0	0	0	0
1	0	0	1	0	0	1	0	0
1	0	1	0	0	0	1	1	0
1	0	1	1	0	0	1	1	1
1	1	0	0	—	—	—	—	—
1	1	0	1	—	—	—	—	—
1	1	1	0	—	—	—	—	—
1	1	1	1	—	—	—	—	—

Volendo calcolare l'equazione minimizzata per y'_0 , costruiamo la seguente mappa di Karnaugh:

$x_1 x_0$ $y_1 y_0$	00	01	11	10
00	0	1	0	0
01	1	0	0	0
11	-	-	-	-
10	0	0	0	0

che permette di individuare l'equazione minimizzata $y'_0 = (x_0 \overline{x_1} y_0 y_1) + (\overline{x_0} x_1 y_0)$, corrispondente al circuito equivalente:



Esercizio 3

Il calcolo delle dimensioni dei campi avviene analizzando la struttura della cache:

- Offset: Determinato dalla dimensione del blocco. Dimensione blocco = $64 \text{ B} = 2^6 \text{ B} \Rightarrow 6 \text{ bit}$
- Index: Determinato dal numero totale di linee (slot) nella cache:

$$\text{Numero di linee} = \frac{\text{Capacità cache}}{\text{Dimensione blocco}} = \frac{64 \text{ KB}}{64 \text{ B}} = \frac{2^{16}}{2^6} = 2^{10} = 1024 \Rightarrow 10 \text{ bit}$$

- Tag: I bit rimanenti dell'indirizzo a 32 bit.

$$\text{Tag} = 32 - (\text{Index} + \text{Offset}) = 32 - (10 + 6) = 16 \text{ bit}$$

Configurazione finale dell'indirizzo:

[Tag: 16 bit | Index: 10 bit | Offset: 6 bit]

Per individuare la linea di cache associata all'indirizzo `0x001A4F3C`, si converte l'indirizzo esadecimale in binario: `0x001A4F3C` $\Rightarrow$ `0000 0000 0001 1010 0100 1111 0011 1100`

Tale rappresentazione viene suddivisa secondo i campi precedentemente individuati:

- Tag (16 bit più significativi): `0000 0000 0001 1010`
- Index (10 bit intermedi): `01 0011 1100`
- Offset (6 bit meno significativi): `11 1100`

Convertendo il campo Index in decimale si individua la linea di cache in cui viene memorizzato il byte:

$$01 \ 0011 \ 1100 \text{ in binario} = 2^8 + 2^5 + 2^4 + 2^3 + 2^2 = 256 + 32 + 16 + 8 + 4 = \mathbf{316}.$$

Progetto

Un processore `z64` controlla il sistema di gestione della clorazione di una piscina. Tale sistema è composto da un dispositivo `MISURAZIONE` costituito da una sonda amperometrica per misurare la concentrazione di cloro libero (rappresentato come word) e il pH dell'acqua (rappresentato come byte). All'avvio del sistema, il processore programma `MISURAZIONE` per acquisire in maniera sincrona i due valori da due appositi registri di interfaccia. Dopodiché, lo `z64` calcola la correzione della misura della concentrazione di cloro, in quanto l'efficacia disinfettante del cloro libero dipende dal livello di pH, secondo la seguente formula:

$$\text{Cloro}_{\text{corretto}} = \frac{\text{Cloro}_{\text{letto}}}{2} - \frac{\text{pH}}{4}$$

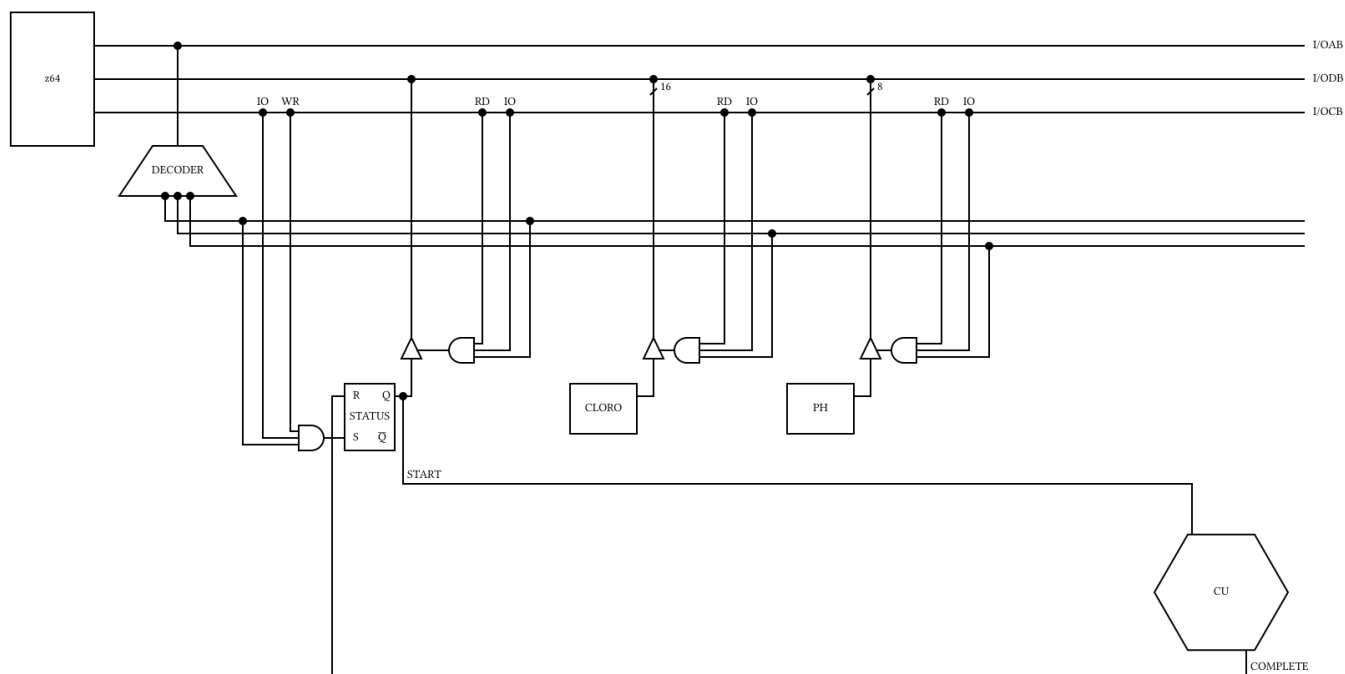
Se il valore di concentrazione di cloro libero corretto è superiore o uguale a 1.5 PPM, non viene innescata alcuna azione aggiuntiva e il sistema continua ad interagire con MISURAZIONE in maniera ciclica. Viceversa, se la concentrazione di cloro è inferiore a 1.5 PPM, il processore avvia il dispositivo CLORATORE per inserire ipoclorito di sodio in piscina per 10 secondi. Al termine di questo intervallo di tempo, CLORATORE avvisa il processore del completamento dell'operazione.

Realizzare:

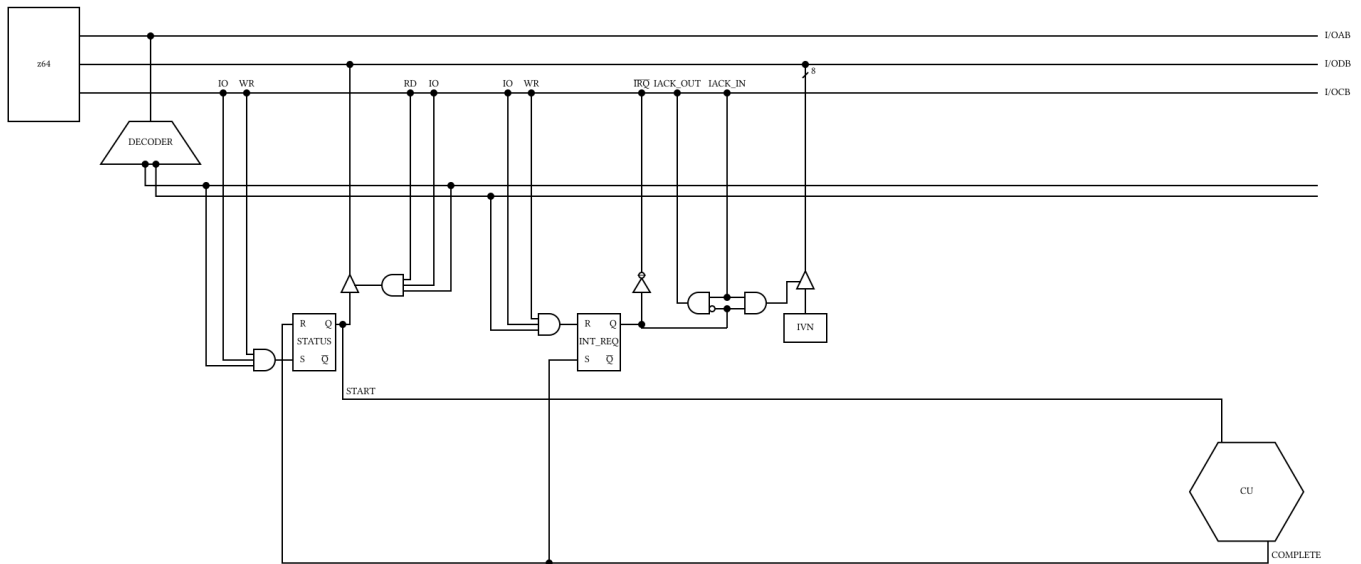
- Le interfacce dei dispositivi MISURAZIONE e CLORATORE ;
- Tutto il codice necessario al funzionamento del sistema, senza l'utilizzo di pseudo-istruzioni.

Soluzione

La periferica MISURAZIONE è un dispositivo sincrono di input dotato di due differenti registri, pertanto la sua interfaccia è la seguente:



La periferica CLORATORE è invece un dispositivo asincrono, ma che non produce né consuma dati:



Assumendo che i valori letti siano in PPM interi, si pone il problema di valutare se il valore letto è maggiore/minore di 1.5, non avendo a disposizione una FPU. Si può applicare una trasformazione algebrica alla condizione, moltiplicando l'intera espressione per 4 ed ottenendo: $2 \cdot \text{Cloro}_{\text{letto}} - \text{pH} < 6$.

Poiché il valore letto del pH potrebbe essere maggiore del doppio del cloro, occorre utilizzare l'aritmetica segnata.

Il codice del sistema può essere il seguente.

```

1  .org 0x800
2  .equ CLORATORE_IRQ, 0x0100
3  .equ CLORATORE_STATUS, 0x0101
4  .equ MISURAZIONE_STATUS, 0x0202
5  .equ MISURAZIONE_CLORO, 0x0204
6  .equ MISURAZIONE_PH, 0x0206
7
8  .data
9  clorazione_in_corso: .byte 0
10
11 .text
12 main:
13     sti
14 .wait:
15     cmpb $1, clorazione_in_corso
16     jz .wait
17
18     outb %al, $MISURAZIONE_STATUS
19 .bw:
20     inb $MISURAZIONE_STATUS, %al
21     testb $1, %al
22     jz .bw
23     inw $MISURAZIONE_CLORO, %ax
24     movw %ax, %bx
25     inb $MISURAZIONE_PH, %al
26     movzbw %al, %cx
27     shlw $1, %bx
28     subw %cx, %bx
29     cmpw $6, %bx
30
31     js .SFset
32     jo .avvia_cloratore # SF=0, OF=1 → SF≠OF (minore)
33     jmp .wait           # SF=0, OF=0 → SF=OF (maggiore o uguale)
34 .SFset:
35     jno .avvia_cloratore # SF=1, OF=0 → SF≠OF (minore)
36     jmp .wait           # SF=1, OF=1 → SF=OF (maggiore o uguale)

```

```
37
38 .avvia_cloratore:
39     movb $1, clorazione_in_corso
40     outb %al, $CLORATORE_STATUS
41     jmp .wait
42
43 .driver 0 # CLORATORE
44     movb $0, clorazione_in_corso
45     outb %al, $CLORATORE_IRQ
46     iret
```